SELENIUM HYBRID FRAMEWORK

Parallel Execution, ThreadLocal, Driver Factory & Retry Mechanism

Complete Beginner-to-Project-Level Notes

Learning outcome Understand how to create browser sessions by configuration, run TestNG tests in parallel without mixing browser sessions, organize page objects cleanly, and retry failed tests responsibly.

Based on the uploaded class flow: Flow(3).txt

Includes source-code walkthroughs, corrected implementations, real-world scenarios, troubleshooting, exercises and interview questions.

How to use these notes

These notes follow the same progression as the class material. First understand the simple browser-selection design, then see why that approach is not sufficient for parallel execution. After that, study ThreadLocal driver management, the BasePage refactor, the Singleton discussion, and finally TestNG retry support.

Label used in these notes	Meaning
Class flow	The concept or structure directly demonstrated in the uploaded source.
Important correction	A defect, incomplete explanation, or misleading point in the source that should not be copied into a production framework.
Recommended implementation	A safer and easier-to-maintain version of the same idea.
Example locator / URL	A teaching example. Confirm it against the current application DOM before using it.

Scope note The source does not include a Maven pom.xml, a complete application DOM, a registration-success locator, or a specific exception stack trace. Therefore dependency versions, success-message selectors and the exact cause of any local exception cannot be established from the source alone.

Contents

- 1. Big picture: what the class is building
- 2. Project structure and responsibilities
- 3. Core concepts in plain language
- 4. Browser selection and Driver Factory
- 5. BaseTest and TestNG browser parameters
- 6. RegisterPage: Page Object Model and PageFactory
- 7. TestNG XML and cross-browser execution
- 8. Why parallel execution breaks with a static driver
- 9. ThreadLocal: thread-specific WebDriver management
- 10. Refactoring BasePage and RegisterPage
- 11. Singleton design pattern: what the source does and does not do
- 12. Retry mechanism with IRetryAnalyzer
- 13. IAnnotationTransformer and XML listener registration
- 14. Recommended end-to-end framework structure
- 15. Real-world scenarios and design choices
- 16. Troubleshooting guide
- 17. Best practices and code-review checklist
- 18. Hands-on exercises
- 19. Interview questions and answers
- 20. Glossary and final revision sheet

1. Big picture: what the class is building

The class evolves a small Selenium + Java + TestNG automation framework. The goal is not only to fill in a registration form. The goal is to build a reusable framework that can choose a browser from XML, create a separate WebDriver session for each parallel TestNG thread, keep page classes independent from test setup, and rerun a failed test a limited number of times.

The complete flow

```
testng.xml
  |
  | supplies browser=chrome / firefox / edge / headless / incognito
  v
RegisterPageTest (TestNG lifecycle)
  |
  | @BeforeTest -> setup(browser)
  v
DriverFactory
  |
  | creates the requested browser session
  v
DriverManagement (ThreadLocal<WebDriver>)
  |
  | stores driver for the current TestNG worker thread
  v
BaseTest / BasePage
  |
  | test layer gets the current driver; page layer uses it
  v
RegisterPage (Page Object)
  |
  | locators + user actions
  v
Application under test

If a test fails:
TestNG -> Transformer -> Retry -> IRetryAnalyzer decides whether to rerun
```

Why these components are separated

Component	Responsibility	Should not do
testng.xml	Provide suite layout, browser parameters, parallel settings and listener registration.	Contain page locators or test business logic.
DriverFactory	Create the correct browser type and browser options.	Store one global mutable driver for all threads.
DriverManagement	Store/retrieve/remove the driver associated with the current thread.	Decide which browser to create.
BaseTest	Own test lifecycle: start browser, configure it, open the application, close it.	Contain page-specific locators.
BasePage	Give page objects access to the current thread driver and common page-level behavior.	Extend the test class or own TestNG lifecycle.
RegisterPage	Represent registration-page elements and	Contain test data or assertions about the

Component	Responsibility	Should not do
	user actions.	test outcome.
Retry / Transformer	Attach retry behavior to TestNG test methods.	Hide genuine application defects by retrying forever.

Key mental model A browser is a resource owned by one test execution thread. In parallel execution, each worker thread needs its own driver. A shared static WebDriver reference can be overwritten by another thread.

2. Project structure and responsibilities

The source asks for Base and Pages under src/main/java, Test under src/test/java, and later a utils package under src/main/java. Java package names conventionally use lowercase, so use base, pages, test or tests, and utils.

```
selenium-hybrid-framework/
|-- pom.xml
|-- testng.xml
|
|-- src/
   |-- main/
   |    |-- java/
   |        |-- base/
   |        |    |-- BaseTest.java
   |        |-- pages/
   |        |    |-- BasePage.java
   |        |    |-- RegisterPage.java
   |        |-- utils/
   |             |-- DriverFactory.java
   |             |-- DriverManagement.java
   |             |-- Retry.java
   |             |-- Transformer.java
   |
   |-- test/
      |-- java/
         |-- tests/
            |-- RegisterPageTest.java
         |-- resources/
            |-- testng.xml  (optional location)
```

Why keep test classes under src/test/java?

- src/main/java is intended for reusable framework/application-support code that production or test code can import.
- src/test/java is intended for test classes. Maven test compilation and test execution treat this source set specially.
- TestNG XML may be kept at the project root or in test resources, depending on the build setup. The path configured in Maven or the IDE must match its actual location.

Naming consistency The source uses package `test` and a folder labelled Test. Choose one lowercase package name such as `tests` and use that exact fully qualified class name in XML: `tests.RegisterPageTest`.

3. Core concepts in plain language

Term	Beginner-friendly explanation	Example
WebDriver	The Selenium control object that sends commands to a browser.	driver.get(url), driver.findElement(...).click().
Browser session	One launched browser instance controlled by one WebDriver.	One Chrome window for one test thread.
Driver Factory	A central object/class that decides which driver implementation to create.	ChromeDriver, EdgeDriver or FirefoxDriver.
POM	Page Object Model: represent each application page as a Java class.	RegisterPage holds first-name locator and enterFirstName().
PageFactory	Selenium support utility that initializes fields annotated with @FindBy.	PageFactory.initElements(driver, this).
TestNG parameter	A value passed from testng.xml into a configuration or test method.	@Parameters("browser") String browser.
Parallel execution	Run multiple test executions at the same time.	Chrome test and Firefox test run concurrently.
ThreadLocal	A Java holder that gives each thread its own value.	Thread A gets Chrome driver; Thread B gets Firefox driver.
Singleton	A design pattern intended to provide one shared instance of a class.	A single DriverManagement manager object.
Retry analyzer	A TestNG extension that decides if a failed test should execute again.	Retry a temporary browser startup failure.
Annotation transformer	A TestNG listener that can modify test annotation metadata before execution.	Attach Retry.class to @Test methods.

A simple analogy

Imagine a testing lab with several operators. Each operator has their own browser. The DriverFactory is the equipment desk that gives an operator the requested browser. ThreadLocal is the labelled locker that ensures each operator retrieves only their own browser. If all operators write their browser into one shared variable, the last write can overwrite the earlier one and commands may go to the wrong browser.

4. Browser selection and Driver Factory

The source first places browser-selection logic inside BaseTest.initialization(String browser). It supports Chrome, Edge, Firefox, headless Chrome, and Chrome incognito mode. The next step extracts that logic into DriverFactory so browser creation lives in one centralized location.

4.1 Initial browser-selection approach from class

```
public static WebDriver driver;

public static void initialization(String browser) {
   if (browser.equalsIgnoreCase("CHROME")) {
      driver = new ChromeDriver();
   } else if (browser.equalsIgnoreCase("EDGE")) {
      driver = new EdgeDriver();
   } else if (browser.equalsIgnoreCase("FIREFOX")) {
      driver = new FirefoxDriver();
   } else if (browser.equalsIgnoreCase("HEADLESS")) {
      ChromeOptions options = new ChromeOptions();
      options.addArguments("--headless");
      driver = new ChromeDriver(options);
   } else if (browser.equalsIgnoreCase("INCOGNITO")) {
      ChromeOptions options = new ChromeOptions();
      options.addArguments("--incognito");
      driver = new ChromeDriver(options);
   } else {
      throw new IllegalStateException("Invalid Browser");
   }
}
```

Line-by-line explanation

- `initialization(String browser)` accepts a browser name from the caller. For example, the XML can pass `chrome`.
- `equalsIgnoreCase` makes `chrome`, `CHROME` and `Chrome` match. It does not handle a null argument; calling a method on null causes NullPointerException.
- `new ChromeDriver()` creates a Chrome browser session. EdgeDriver and FirefoxDriver create their respective sessions.
- `ChromeOptions` is a browser-specific configuration object. It is passed to ChromeDriver constructor.
- `--headless` asks Chrome to run without a visible browser UI. Headless mode is useful in CI environments without a desktop display.
- `--incognito` starts Chrome with an incognito profile. It does not make automation anonymous or secure credentials.
- The `else` branch fails fast for an unsupported browser rather than silently choosing a default.

Important correction The initial version stores the driver in a static field. That is acceptable only for a simple single-threaded demonstration. It is unsafe when multiple tests run at the same time.

4.2 Extracting DriverFactory

A factory centralizes object creation. The first DriverFactory in the uploaded source returns a WebDriver from setDriver(browser). Despite the method name `setDriver`, returning a driver is clearer when the method is named `createDriver`.

```
package utils;

import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;
import org.openqa.selenium.chrome.ChromeOptions;
import org.openqa.selenium.edge.EdgeDriver;
import org.openqa.selenium.firefox.FirefoxDriver;

public final class DriverFactory {
   private DriverFactory() { }

   public static WebDriver createDriver(String browser) {
      if (browser == null || browser.isBlank()) {
         throw new IllegalArgumentException("Browser name must not be empty");
      }

      if (browser.equalsIgnoreCase("chrome")) {
         return new ChromeDriver();
      } else if (browser.equalsIgnoreCase("edge")) {
         return new EdgeDriver();
      } else if (browser.equalsIgnoreCase("firefox")) {
         return new FirefoxDriver();
      } else if (browser.equalsIgnoreCase("headless")) {
         ChromeOptions options = new ChromeOptions();
         options.addArguments("--headless");
         return new ChromeDriver(options);
      } else if (browser.equalsIgnoreCase("incognito")) {
         ChromeOptions options = new ChromeOptions();
         options.addArguments("--incognito");
         return new ChromeDriver(options);
      }

      throw new IllegalArgumentException("Unsupported browser: " + browser);
   }
}
```

What improved?

- The method name tells you what it does: it creates and returns a WebDriver.
- It validates null or blank browser values before calling equalsIgnoreCase.
- It is a utility class with a private constructor, so callers do not need to create DriverFactory objects.
- The `return` statements make it impossible to accidentally forget to return the selected driver in a valid branch.
- An IllegalArgumentException is suitable when a caller supplies an invalid argument.

4.3 Browser modes compared

XML value	Driver created	Purpose / limitations
chrome	ChromeDriver	Normal headed Chrome session.
edge	EdgeDriver	Normal Microsoft Edge session.
firefox	FirefoxDriver	Normal Firefox session.
headless	ChromeDriver + ChromeOptions	Runs Chrome without a visible window. Screenshot debugging may still be possible, but visual behavior can differ from headed mode.
incognito	ChromeDriver + --incognito	Uses an incognito browser window. It still runs a normal browser process and still

XML value	Driver created	Purpose / limitations
		sends network traffic.

Environment requirement Selenium Manager can assist with driver management in modern Selenium releases, but the installed browser, Selenium version, network access and corporate restrictions still matter. If startup fails, inspect the full exception and browser/driver compatibility rather than assuming the factory branch is wrong.

5. BaseTest and TestNG browser parameters

After extracting driver creation, BaseTest asks DriverFactory for the driver and then configures the browser. The class source sets implicit wait, page-load timeout, maximizes the window and opens the Guru99 New Tours registration URL.

5.1 The source version

```
public class BaseTest {
    public static WebDriver driver;

    public static void initialization(String browser) {
        driver = DriverFactory.setDriver(browser);
        driver.manage().timeouts()
              .implicitlyWait(Duration.ofSeconds(30));
        driver.manage().timeouts()
              .pageLoadTimeout(Duration.ofSeconds(30));
        driver.manage().window().maximize();
        driver.get("https://demo.guru99.com/test/newtours/register.php");
    }
}
```

In the next class steps, `DriverFactory.setDriver` is changed to return void and store the created object in ThreadLocal through DriverManagement. BaseTest then retrieves it using DriverManagement.getDriver(). That change is explained in the ThreadLocal chapter.

5.2 Recommended lifecycle ownership

A framework is easier to maintain when the test base class owns setup and teardown. The source puts @BeforeTest and @AfterTest directly in RegisterPageTest. That works for a small example, but a BaseTest class allows many test classes to reuse the same lifecycle logic.

```
package base;

import java.time.Duration;
import org.openqa.selenium.WebDriver;
import org.testng.annotations.AfterTest;
import org.testng.annotations.BeforeTest;
import org.testng.annotations.Parameters;
import utils.DriverFactory;
import utils.DriverManagement;

public class BaseTest {
   @BeforeTest
   @Parameters("browser")
   public void setUp(String browser) {
      WebDriver driver = DriverFactory.createDriver(browser);
      DriverManagement.setDriver(driver);

      driver.manage().timeouts()
            .implicitlyWait(Duration.ZERO);
      driver.manage().timeouts()
            .pageLoadTimeout(Duration.ofSeconds(30));
      driver.manage().window().maximize();
      driver.get("https://demo.guru99.com/test/newtours/register.php");
   }

   @AfterTest(alwaysRun = true)
   public void tearDown() {
      WebDriver driver = DriverManagement.getDriverOrNull();
      try {
         if (driver != null) {
            driver.quit();
         }
      } finally {
         DriverManagement.unload();
      }
   }
}
```

Why use these lifecycle methods?

Annotation	When it runs	Use here
@BeforeTest	Before the test methods belonging to a TestNG <test> tag.	Read browser parameter and create a session for that TestNG test context.
@Test	Marks a method as an executable test.	Perform registration actions and assert the result.
@AfterTest	After test methods belonging to that TestNG <test> tag.	Quit the browser and clear thread-local state.
alwaysRun = true	Allows the teardown configuration to run even when earlier methods failed or were skipped, subject to TestNG lifecycle rules.	Improve cleanup reliability.

Implicit wait recommendation The source uses a 30-second implicit wait. In a framework that uses explicit waits, prefer setting implicit wait to zero (or keeping it consistently minimal). Mixing implicit and explicit waits can create confusing compounded wait times.

5.3 @Parameters explained

```
@Parameters("browser")
@BeforeTest
public void setUp(String browser) {
    BaseTest.initialization(browser);
}
```

`@Parameters("browser")` tells TestNG to inject the XML parameter named browser into the Java method argument. The spelling must match exactly. If XML says `browser` but Java expects `browserName`, that is fine only if the annotation still names `browser`; the Java variable name itself can differ.

6. RegisterPage: Page Object Model and PageFactory

RegisterPage is a Page Object. It holds the web elements belonging to the registration page and exposes methods that represent user actions. Test classes should call these methods rather than duplicate XPath or CSS locators.

6.1 The source locator repository

```
@FindBy(name = "firstName")
WebElement firstName;

@FindBy(name = "lastName")
WebElement lastName;

@FindBy(name = "phone")
WebElement phone;

@FindBy(name = "userName")
WebElement email;

@FindBy(name = "address1")
WebElement address;

@FindBy(name = "city")
WebElement city;

@FindBy(name = "state")
WebElement state;

@FindBy(name = "postalCode")
WebElement postalCode;

@FindBy(name = "country")
WebElement country;

@FindBy(name = "email")
WebElement username;

@FindBy(name = "password")
WebElement password;

@FindBy(name = "confirmPassword")
WebElement confirmPassword;

@FindBy(name = "submit")
WebElement submit;
```

What does @FindBy do?

`@FindBy` declares how PageFactory should locate a WebElement. For example, `@FindBy(name = "firstName")` is equivalent in locator meaning to `By.name("firstName")`. The browser DOM must contain an element with that name attribute. The annotation does not prove the locator is correct; the locator must be checked against the application HTML.

What does PageFactory.initElements do?

```
public RegisterPage() {
    PageFactory.initElements(driver, this);
}
```

- `driver` is the WebDriver used to search the current page.
- `this` refers to the current RegisterPage object being constructed.
- PageFactory inspects fields annotated with FindBy and prepares the WebElement proxies.
- When a page method interacts with a proxy, Selenium resolves the element using its locator.

Locator naming issue in source The source maps `name="userName"` to a Java field named email, and maps `name="email"` to a field named username. The locator attributes may be valid, but the Java field names are confusing. Rename fields to reflect their actual form meaning after inspecting the page labels.

6.2 Recommended RegisterPage implementation

The following version keeps the same fields and actions from the class, improves visibility, uses clear field names, and keeps the page object separate from test lifecycle code. The locator attributes remain the ones supplied in the class and must be validated against the actual page.

```
package pages;

import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.support.FindBy;
import org.openqa.selenium.support.PageFactory;
import org.openqa.selenium.support.ui.Select;
import utils.DriverManagement;

public class RegisterPage {
    private final WebDriver driver;

    @FindBy(name = "firstName")
    private WebElement firstName;

    @FindBy(name = "lastName")
    private WebElement lastName;

    @FindBy(name = "phone")
    private WebElement phone;

    @FindBy(name = "userName")
    private WebElement emailAddress;

    @FindBy(name = "address1")
    private WebElement address;

    @FindBy(name = "city")
    private WebElement city;

    @FindBy(name = "state")
    private WebElement state;

    @FindBy(name = "postalCode")
    private WebElement postalCode;

    @FindBy(name = "country")
    private WebElement countryDropdown;

    @FindBy(name = "email")
    private WebElement accountUsername;

    @FindBy(name = "password")
    private WebElement password;

    @FindBy(name = "confirmPassword")
    private WebElement confirmPassword;

    @FindBy(name = "submit")
    private WebElement submitButton;

    public RegisterPage() {
        this.driver = DriverManagement.getDriver();
        PageFactory.initElements(driver, this);
    }

    public void enterFirstName(String value) { firstName.sendKeys(value); }
    public void enterLastName(String value) { lastName.sendKeys(value); }
    public void enterPhone(String value) { phone.sendKeys(value); }
    public void enterEmailAddress(String value) { emailAddress.sendKeys(value); }
    public void enterAddress(String value) { address.sendKeys(value); }
    public void enterCity(String value) { city.sendKeys(value); }
    public void enterState(String value) { state.sendKeys(value); }
    public void enterPostalCode(String value) { postalCode.sendKeys(value); }

    public void selectCountry(String visibleText) {
```

```
        new Select(countryDropdown).selectByVisibleText(visibleText);
    }

    public void enterAccountUsername(String value) { accountUsername.sendKeys(value); }
    public void enterPassword(String value) { password.sendKeys(value); }
    public void enterConfirmPassword(String value) { confirmPassword.sendKeys(value); }
    public void clickSubmit() { submitButton.click(); }
}
```

6.3 Understanding Select

```
public void selectCountry(String visibleText) {
    Select countrySelect = new Select(countryDropdown);
    countrySelect.selectByVisibleText(visibleText);
}
```

Selenium Select is used for a real HTML `<select>` element. `selectByVisibleText("INDIA")` selects the option whose visible label is INDIA. If the website uses a custom dropdown built from div, span or button elements, Selenium Select will throw an exception; interact with the custom dropdown like ordinary elements instead.

6.4 Why methods accept parameters

Page methods should describe an action, not hard-code test data. `enterFirstName(String value)` can be reused for valid, invalid, boundary and data-driven test cases. The test class decides which value to pass.

7. TestNG XML and cross-browser execution

The class converts the test class to a TestNG XML file, then supplies a browser parameter. The test method does not need to change when the browser changes; only the XML parameter changes.

7.1 Single-browser XML from the source

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE suite SYSTEM "https://testng.org/testng-1.0.dtd">

<suite name="Suite">
  <test name="RegisterOnChrome">
    <parameter name="browser" value="chrome"/>
    <classes>
      <class name="tests.RegisterPageTest"/>
    </classes>
  </test>
</suite>
```

When TestNG runs this XML, it finds the test class, reads the parameter, and injects the value into a configuration method annotated with @Parameters("browser"). The package and class name in XML must match the actual Java package and class declaration.

7.2 Cross-browser serial execution

```
<suite name="CrossBrowserSuite">
 <test name="ChromeRun">
  <parameter name="browser" value="chrome"/>
  <classes><class name="tests.RegisterPageTest"/></classes>
 </test>

 <test name="FirefoxRun">
  <parameter name="browser" value="firefox"/>
  <classes><class name="tests.RegisterPageTest"/></classes>
 </test>
</suite>
```

Without a parallel attribute, the TestNG test groups run sequentially by default. This is useful when you want to check the same test flow on different browsers but do not yet need parallel speed.

7.3 Parallel XML

```
<suite name="CrossBrowserParallelSuite"
      parallel="tests"
      thread-count="2">

 <test name="ChromeRun">
  <parameter name="browser" value="chrome"/>
  <classes><class name="tests.RegisterPageTest"/></classes>
 </test>

 <test name="FirefoxRun">
  <parameter name="browser" value="firefox"/>
  <classes><class name="tests.RegisterPageTest"/></classes>
 </test>
</suite>
```

What does parallel="tests" mean?

- `<suite>` is the top-level suite definition.
- Each `<test>` tag is a TestNG test context. With parallel="tests", TestNG can execute separate `<test>` contexts concurrently.
- `thread-count="2"` allows up to two worker threads for these parallel test contexts.
- The ChromeRun context receives browser=chrome, while FirefoxRun receives browser=firefox.
- Each context must create, use and close its own driver session.

Do not confuse TestNG <test> with @Test The XML `<test>` element groups classes and parameters. The Java `@Test` annotation marks a test method. They are related, but they are not the same thing.

7.4 What other parallel modes mean

parallel value	Meaning	Typical use
tests	Run each XML <test> context in parallel.	Run a class in Chrome and another context in Firefox.
classes	Run test classes in parallel.	Different feature classes run concurrently.
methods	Run individual @Test methods in parallel.	Independent test methods execute at the

parallel value	Meaning	Typical use
		same time.
instances	Run test class instances in parallel.	Useful when object instances represent isolated test contexts.

8. Why parallel execution breaks with a static driver

The source asks why the XML with `parallel="tests"` throws an exception. The uploaded material does not include the actual stack trace, so the exact exception cannot be diagnosed. However, the first implementation has a clear concurrency defect: the WebDriver is held in a single static field.

8.1 The shared-static-driver race

```
public class BaseTest {
   public static WebDriver driver;
}
```

A static variable belongs to the class, not to an individual test thread. Consider this possible execution timeline:

Time	Chrome worker thread	Firefox worker thread	Shared static driver
T1	Creates ChromeDriver	-	ChromeDriver
T2	-	Creates FirefoxDriver	FirefoxDriver (overwrites Chrome reference)
T3	Calls driver.findElement(...)	-	Both threads may now read FirefoxDriver
T4	May interact with wrong session or fail	Uses FirefoxDriver	FirefoxDriver

Likely symptoms Tests may type into the wrong browser, close another test's session, fail with NoSuchWindowException or NoSuchSessionException, or behave inconsistently. The exact symptom depends on timing.

8.2 Parallelism does not create isolation automatically

- TestNG creates worker threads; it does not automatically make your static variables thread-safe.
- The factory can create two drivers correctly and the framework can still fail if both threads store the driver in one shared variable.
- Page object static fields can introduce the same class of race. A ThreadLocal driver alone does not make static page fields safe.
- Reusing the same email address in two simultaneous registration flows can also cause data collisions, even after driver handling is fixed.

8.3 Other possible XML/setup errors

Possible issue	What to verify
ParameterException / missing parameter	The XML parameter name matches @Parameters("browser") and the configuration method signature.
Invalid browser exception	The XML value exactly matches a supported factory option, ignoring case.
NullPointerException	Driver creation happened before getDriver(), browser argument is null, or initialization failed.
NoSuchSessionException	Driver was already quit, shared across threads, or accessed after teardown.
Duplicate registration failure	Parallel tests use the same account/email data; generate unique data per execution.
Element not found / timeout	The DOM locator is valid, the correct page loaded, and synchronization is appropriate.
Driver/browser startup failure	Browser installed, permissions available, Selenium version and browser support aligned, CI display/headless flags correct.

Diagnostic request To identify the exact exception, the stack trace and the first line of the exception are required. The source only shows the XML and framework code, not the runtime failure.

9. ThreadLocal: thread-specific WebDriver management

ThreadLocal is a Java class that provides a separate value for each thread. When TestNG runs two parallel test contexts, each context runs on a worker thread. Each thread can store and retrieve its own WebDriver without overwriting the other thread's value.

9.1 ThreadLocal analogy

Think of two employees sharing the same office but having separate lockers. Employee A stores the Chrome key in Locker A. Employee B stores the Firefox key in Locker B. When each employee asks for their key, they get only the key stored in their own locker. ThreadLocal behaves similarly for values associated with a Java thread.

9.2 DriverManagement from source

```
package utils;

import org.openqa.selenium.WebDriver;

public class DriverManagement {
   private static ThreadLocal<WebDriver> threadLocal =
         new ThreadLocal<WebDriver>();

   public static void setDriver(WebDriver driverName) {
      threadLocal.set(driverName);
   }

   public static WebDriver getDriver() {
      return threadLocal.get();
   }
}
```

Understand each operation

- `ThreadLocal<WebDriver>` means each thread stores a WebDriver value.
- `new ThreadLocal<>()` creates the container. The generic type tells Java that the stored value must be a WebDriver reference.
- `set(driver)` saves the driver for the currently executing thread.
- `get()` returns the driver stored for the currently executing thread. If none was stored, get() returns null.
- A different thread calling get() sees its own value, not the value set by another thread.

9.3 Recommended DriverManagement

```
package utils;

import java.util.Objects;
import org.openqa.selenium.WebDriver;

public final class DriverManagement {
   private static final ThreadLocal<WebDriver> DRIVER =
         new ThreadLocal<>();

   private DriverManagement() { }

   public static void setDriver(WebDriver driver) {
      DRIVER.set(Objects.requireNonNull(driver,
            "WebDriver must not be null"));
   }

   public static WebDriver getDriver() {
      WebDriver driver = DRIVER.get();
      if (driver == null) {
         throw new IllegalStateException(
               "No WebDriver is stored for this thread. " +
               "Call setDriver() before using a page object.");
      }
      return driver;
   }

   public static WebDriver getDriverOrNull() {
      return DRIVER.get();
   }

   public static void unload() {
      DRIVER.remove();
   }
}
```

Why is remove() important?

TestNG and many execution frameworks use thread pools. A worker thread may be reused for another test after its current test finishes. Calling `ThreadLocal.remove()` during teardown removes the value associated with that worker. It prevents an old driver reference from remaining attached to a reused thread and makes lifecycle ownership explicit.

Critical lifecycle rule Create the driver -> set it in ThreadLocal -> use it -> quit it -> call ThreadLocal.remove(). Do not quit the driver but leave its reference in ThreadLocal.

9.4 ThreadLocal does not mean multiple threads share one driver

The exact purpose is the opposite: it allows multiple threads to hold different driver references under the same static ThreadLocal variable. A single ThreadLocal field can manage many per-thread values.

10. Refactoring BasePage and RegisterPage

The class first makes RegisterPage extend BaseTest. Later, it changes the design so RegisterPage extends BasePage instead. This is an important framework design improvement: page objects should not inherit test setup and teardown behavior.

10.1 Why not extend BaseTest from every page?

- BaseTest is a test-layer class. It owns TestNG setup, teardown, and browser lifecycle.
- RegisterPage is a page-layer class. It represents one page and its actions.
- If every page inherits BaseTest, page classes become coupled to test lifecycle code and can accidentally access shared test state.
- A BasePage is a better place for common page-level dependencies such as the current WebDriver.

10.2 BasePage from source and recommended correction

```
package pages;

import org.openqa.selenium.WebDriver;
import utils.DriverManagement;

public class BasePage {
   protected final WebDriver driver;

   public BasePage() {
      this.driver = DriverManagement.getDriver();
   }
}
```

The source declares `protected static WebDriver driver`. This is a major issue in the parallel version. A static BasePage.driver is still shared by every page object in the JVM. Each BasePage construction can overwrite it. Use an instance field (`protected final WebDriver driver`) rather than static.

10.3 RegisterPage after refactor

```
public class RegisterPage extends BasePage {
   @FindBy(name = "firstName")
   private WebElement firstName;

   @FindBy(name = "lastName")
   private WebElement lastName;

   @FindBy(name = "submit")
   private WebElement submitButton;

   public RegisterPage() {
      PageFactory.initElements(driver, this);
   }

   public void enterFirstName(String value) {
      firstName.sendKeys(value);
   }

   public void enterLastName(String value) {
      lastName.sendKeys(value);
   }

   public void clickSubmit() {
      submitButton.click();
   }
}
```

Here, `driver` is inherited from BasePage as a protected instance field. Since each page object receives the current thread's driver when it is created, parallel test threads do not overwrite a single global page driver.

10.4 Object lifecycle example

```
Thread A: DriverManagement.setDriver(chromeDriver)
Thread A: new RegisterPage() -> BasePage gets chromeDriver
Thread A: RegisterPage fields initialized with chromeDriver

Thread B: DriverManagement.setDriver(firefoxDriver)
Thread B: new RegisterPage() -> BasePage gets firefoxDriver
Thread B: RegisterPage fields initialized with firefoxDriver
```

Important Create page objects after the driver has been created and stored in DriverManagement. Creating RegisterPage too early will produce an initialization error or null driver.

11. Singleton design pattern: source code review

The class labels this change “Singleton Design Pattern with ThreadLocal class”. The important point is that the code shown does not actually implement the Singleton pattern.

11.1 Singleton version from source

```
public class DriverManagement {
   private static ThreadLocal<WebDriver> threadLocal =
         new ThreadLocal<WebDriver>();

   private DriverManagement() {
   }

   public static DriverManagement getInstance() {
      return new DriverManagement();
   }

   public static void setDriver(WebDriver driverName) {
      threadLocal.set(driverName);
   }

   public WebDriver getDriver() {
      return threadLocal.get();
   }
}
```

Why is this not a Singleton?

A Singleton must return the same instance every time getInstance() is called. The source executes `new DriverManagement()` inside getInstance(), so every call creates a new object. The private constructor prevents callers from using `new DriverManagement()` directly, but it does not make getInstance() return a single shared instance.

11.2 Correct Singleton implementation (educational example)

```
public final class DriverManagement {
   private static final DriverManagement INSTANCE =
         new DriverManagement();

   private static final ThreadLocal<WebDriver> DRIVER =
         new ThreadLocal<>();

   private DriverManagement() { }

   public static DriverManagement getInstance() {
      return INSTANCE;
   }

   public void setDriver(WebDriver driver) {
      DRIVER.set(driver);
   }

   public WebDriver getDriver() {
      return DRIVER.get();
   }

   public void unload() {
      DRIVER.remove();
   }
}
```

This returns the same manager object every time. The manager object is shared, but its ThreadLocal field still stores a different WebDriver for each calling thread.

11.3 Do we need Singleton here?

Not necessarily. Because the ThreadLocal itself is static, a simple utility class with static methods is enough. Singleton adds an object instance without giving much benefit in this use case. Choose one approach consistently; do not mix static and instance APIs confusingly.

Approach	What it does	Recommendation
Static utility + static ThreadLocal	One utility class manages the per-thread driver values.	Simple and suitable for this small framework.
Real Singleton + static ThreadLocal	One manager instance; driver storage remains per-thread.	Valid for learning Singleton, but may be unnecessary.
Source getInstance() returns new object	Creates a different manager object each call.	Not a Singleton; do not describe it as one.

12. Retry mechanism with IRetryAnalyzer

The source introduces a Retry class because tests may fail intermittently. The Retry class implements TestNG's IRetryAnalyzer interface. TestNG asks the analyzer whether a failed test should be executed again.

12.1 Retry class from source

```
package utils;

import org.testng.IRetryAnalyzer;
import org.testng.ITestResult;

public class Retry implements IRetryAnalyzer {
   int counter = 0;
   int maxCounter = 4;

   @Override
   public boolean retry(ITestResult result) {
      if (counter < maxCounter) {
         counter++;
         return true;
      }
      return false;
   }
}
```

Understand the logic

1. The test method executes for the first time.
2. If it fails, TestNG calls retry(ITestResult result).
3. The analyzer checks whether counter is below maxCounter.
4. If counter is below 4, it increments the counter and returns true. Returning true instructs TestNG to retry the test.
5. After four retries have been granted, the next failure reaches the analyzer with counter equal to 4. It returns false, so no more retries are scheduled.

Important counting correction `maxCounter = 4` means up to FOUR RETRIES after the initial execution. The maximum number of attempts is therefore FIVE (1 initial + 4 retries), not four total attempts.

12.2 Attempt timeline

Attempt	What happens	Retry decision after failure
1	Initial test execution fails.	true -> retry 1
2	First retry fails.	true -> retry 2
3	Second retry fails.	true -> retry 3
4	Third retry fails.	true -> retry 4
5	Fourth retry fails.	false -> stop; final result remains failed

12.3 Improved Retry implementation

```
package utils;

import org.testng.IRetryAnalyzer;
import org.testng.ITestResult;

public class Retry implements IRetryAnalyzer {
   private int retryCount = 0;
   private static final int MAX_RETRIES = 2;

   @Override
   public boolean retry(ITestResult result) {
      if (retryCount < MAX_RETRIES) {
         retryCount++;
         System.out.printf(
            "Retrying %s.%s (retry %d of %d)%n",
            result.getTestClass().getName(),
            result.getMethod().getMethodName(),
            retryCount,
            MAX_RETRIES
         );
         return true;
      }
      return false;
   }
}
```

The example uses two retries to keep local learning runs shorter. Adjust the count based on your project policy. The counter should normally be an instance field rather than static so unrelated test methods do not share one global retry count.

12.4 When should a test be retried?

Potentially retryable	Usually not retryable
Temporary browser startup issue.	A deterministic assertion failure because the feature is broken.
Brief network interruption outside the application.	Wrong expected result or incorrect business rule.
Temporary Grid node/session allocation problem.	Invalid test data or a consistently bad locator.
Known transient service unavailability, when policy allows.	A reproducible application defect that must be reported immediately.

Do not use retry to hide flaky tests A retry can turn a failed first attempt into a passed final result. Track retry counts and investigate flaky causes. Retries should be limited, visible in reports, and used only for approved transient failures.

13. IAnnotationTransformer and XML listener registration

The source creates a Transformer class implementing IAnnotationTransformer. It attaches Retry.class to each test annotation, so you do not need to write `retryAnalyzer = Retry.class` on every @Test method.

13.1 Transformer from source

```
package utils;

import java.lang.reflect.Constructor;
import java.lang.reflect.Method;
import org.testng.IAnnotationTransformer;
import org.testng.annotations.ITestAnnotation;

public class Transformer implements IAnnotationTransformer {
   @Override
   public void transform(ITestAnnotation annotation,
                         Class testClass,
                         Constructor testConstructor,
                         Method testMethod) {
      annotation.setRetryAnalyzer(Retry.class);
   }
}
```

Explain each part

- `IAnnotationTransformer` is a TestNG listener interface that allows a test annotation to be changed before TestNG executes the test.
- `ITestAnnotation` represents metadata for a TestNG @Test method.
- `transform(...)` is called by TestNG while reading test annotations.
- `annotation.setRetryAnalyzer(Retry.class)` tells TestNG which retry analyzer should control failed attempts.

13.2 Register Transformer in testng.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE suite SYSTEM "https://testng.org/testng-1.0.dtd">

<suite name="ParallelSuite" parallel="tests" thread-count="2">
 <listeners>
  <listener class-name="utils.Transformer"/>
 </listeners>

 <test name="ChromeRun">
  <parameter name="browser" value="chrome"/>
  <classes><class name="tests.RegisterPageTest"/></classes>
 </test>

 <test name="FirefoxRun">
  <parameter name="browser" value="firefox"/>
  <classes><class name="tests.RegisterPageTest"/></classes>
 </test>
</suite>
```

The listener class name must be fully qualified and must be available on the test runtime classpath. The Transformer class should have a public no-argument constructor (the implicit default constructor is sufficient if none is declared).

13.3 Apply retry to selected tests only

The source transformer applies retry behavior to every @Test annotation it sees. In a larger suite, this may be too broad. A better policy is to opt in only selected methods or use a marker annotation/configuration policy. At a minimum, document which tests are allowed to retry and why.

Version note The exact IAnnotationTransformer method signature is defined by the TestNG version on the project classpath. Keep the interface method signature aligned with that dependency. The uploaded source does not state the

TestNG version, so no specific version compatibility claim is made here.

14. Recommended end-to-end framework structure

This chapter combines the concepts into one maintainable flow. It deliberately avoids a static global WebDriver and avoids a static BasePage driver. It also keeps browser creation separate from thread-local storage.

14.1 DriverFactory

```
package utils;

import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;
import org.openqa.selenium.chrome.ChromeOptions;
import org.openqa.selenium.edge.EdgeDriver;
import org.openqa.selenium.firefox.FirefoxDriver;

public final class DriverFactory {
   private DriverFactory() { }

   public static WebDriver createDriver(String browser) {
      if (browser == null || browser.isBlank()) {
         throw new IllegalArgumentException("Browser is required");
      }

      switch (browser.trim().toLowerCase()) {
         case "chrome":
            return new ChromeDriver();
         case "edge":
            return new EdgeDriver();
         case "firefox":
            return new FirefoxDriver();
         case "headless":
            ChromeOptions headless = new ChromeOptions();
            headless.addArguments("--headless");
            return new ChromeDriver(headless);
         case "incognito":
            ChromeOptions incognito = new ChromeOptions();
            incognito.addArguments("--incognito");
            return new ChromeDriver(incognito);
         default:
            throw new IllegalArgumentException(
               "Unsupported browser: " + browser);
      }
   }
}
```

14.2 DriverManagement

```
package utils;

import org.openqa.selenium.WebDriver;

public final class DriverManagement {
   private static final ThreadLocal<WebDriver> DRIVER =
         new ThreadLocal<>();

   private DriverManagement() { }

   public static void setDriver(WebDriver driver) {
      if (driver == null) {
         throw new IllegalArgumentException("Driver must not be null");
      }
      DRIVER.set(driver);
   }

   public static WebDriver getDriver() {
      WebDriver driver = DRIVER.get();
      if (driver == null) {
         throw new IllegalStateException("Driver not initialized for this thread");
      }
      return driver;
   }

   public static WebDriver getDriverOrNull() {
      return DRIVER.get();
   }

   public static void unload() {
      DRIVER.remove();
   }
}
```

14.3 BaseTest

```
package base;

import java.time.Duration;
import org.openqa.selenium.WebDriver;
import org.testng.annotations.AfterTest;
import org.testng.annotations.BeforeTest;
import org.testng.annotations.Parameters;
import utils.DriverFactory;
import utils.DriverManagement;

public class BaseTest {
   @BeforeTest
   @Parameters("browser")
   public void setUp(String browser) {
      WebDriver driver = DriverFactory.createDriver(browser);
      DriverManagement.setDriver(driver);

      driver.manage().timeouts().implicitlyWait(Duration.ZERO);
      driver.manage().timeouts().pageLoadTimeout(Duration.ofSeconds(30));
      driver.manage().window().maximize();
      driver.get("https://demo.guru99.com/test/newtours/register.php");
   }

   @AfterTest(alwaysRun = true)
   public void tearDown() {
      WebDriver driver = DriverManagement.getDriverOrNull();
      try {
         if (driver != null) {
            driver.quit();
         }
      } finally {
         DriverManagement.unload();
      }
   }
}
```

This example assumes each XML <test> context gets its own setup/teardown lifecycle. If you instead parallelize individual methods or share a class instance across methods, revisit the lifecycle scope and consider @BeforeMethod/@AfterMethod so each test method owns an isolated browser session.

14.4 BasePage and RegisterPage

```
package pages;

import org.openqa.selenium.WebDriver;
import utils.DriverManagement;

public abstract class BasePage {
   protected final WebDriver driver;

   protected BasePage() {
      this.driver = DriverManagement.getDriver();
   }
}
```

RegisterPage extends BasePage, declares private @FindBy elements, calls PageFactory.initElements(driver, this) from its constructor, and exposes methods such as enterFirstName(), selectCountry() and clickSubmit().

14.5 RegisterPageTest

```
package tests;

import org.testng.Assert;
import org.testng.annotations.Test;
import base.BaseTest;
import pages.RegisterPage;

public class RegisterPageTest extends BaseTest {
   @Test
   public void registerNewUser() {
      RegisterPage page = new RegisterPage();

      page.enterFirstName("Alex");
      page.enterLastName("Tester");
      page.enterPhone("9000000001");
      page.enterEmailAddress("alex.tester+run001@example.com");
      page.enterAddress("10 Test Street");
      page.enterCity("Bengaluru");
      page.enterState("Karnataka");
      page.enterPostalCode("560001");
      page.selectCountry("INDIA");
      page.enterAccountUsername("alex.tester+run001@example.com");
      page.enterPassword(System.getenv("TEST_USER_PASSWORD"));
      page.enterConfirmPassword(System.getenv("TEST_USER_PASSWORD"));
      page.clickSubmit();

      // Replace this with a real success condition from the AUT.
      Assert.assertTrue(page.isRegistrationSuccessful(),
            "Registration confirmation was not displayed");
   }
}
```

Adaptation required `isRegistrationSuccessful()` is an illustrative page method, not present in the uploaded source. Implement it with a locator verified against the application's real success page. A test that clicks Submit but does not assert an observable result is not a complete functional test.

14.6 Teardown improvement

If driver creation throws before ThreadLocal.setDriver(), getDriverOrNull() returns null and teardown can safely continue. If quitting throws, the finally block still removes the ThreadLocal value. For robust reporting, teams can log quit failures without masking the original test failure.

15. Real-world scenarios and design choices

Scenario 1: Smoke test must run on Chrome and Firefox

A team wants to verify that the registration flow works on two browsers for every release. Configure two XML <test> contexts, each with its own browser parameter. Start serially first; after the test is stable and driver isolation is correct, enable parallel="tests" and thread-count="2".

Design decision	Recommended choice
Browser configuration	XML parameter per TestNG <test>.
Driver creation	DriverFactory creates the browser.

Design decision	Recommended choice
Driver storage	ThreadLocal per worker thread.
Data	Unique email per browser run.
Assertion	Verify a real confirmation message or next-page state.
Cleanup	Quit the thread's driver and remove ThreadLocal value.

Scenario 2: CI pipeline runs headless Chrome

A Jenkins or GitHub Actions agent runs tests without a desktop session. Set browser=headless in the XML. Use logs and screenshots on failure. Do not assume headless mode removes the need for correct browser dependencies, permissions or network access.

Scenario 3: Suite contains 80 independent tests

A team wants faster feedback. It can split tests across worker threads, but the thread count should be selected based on machine capacity, Selenium Grid capacity, application environment capacity and test data isolation. More threads do not always mean faster execution; they can overload the environment.

Scenario 4: One test fails because of temporary Grid unavailability

A limited retry may be reasonable if the failure is recognized as a transient infrastructure issue. The retry count should be low and reported. If the same test fails consistently due to a wrong locator or product bug, retrying only wastes execution time.

Scenario 5: Duplicate account data under parallel execution

Two threads attempt to register the same username at the same time. Even with separate browsers and ThreadLocal, one registration may fail because the username is already taken. Generate unique test identities using a run identifier or a data factory. Do not confuse an application data collision with a ThreadLocal defect.

Scenario 6: Browser closes but the worker thread remains alive

A TestNG worker thread may be reused by a thread pool. If ThreadLocal.remove() is not called, a stale reference can remain attached to that worker. Always remove the ThreadLocal value after quitting the browser.

Scenario 7: A custom country dropdown fails with Select

Selenium Select supports native HTML select tags only. Inspect the DOM. If the dropdown is custom, click the control, wait for the list options, and click the desired option using normal WebElement locators.

16. Troubleshooting guide

Error / symptom	Likely reason	Action
NullPointerException at equalsIgnoreCase	browser argument is null.	Validate null/blank before comparing browser names.
Invalid Browser	Value not recognized by factory.	Check XML value and supported names: chrome, edge, firefox, headless, incognito.
Driver is null in RegisterPage	Page created before factory/ThreadLocal	Ensure @BeforeTest or @BeforeMethod ran

Error / symptom	Likely reason	Action
	setup.	before creating the page object.
Two tests control one browser	Shared static WebDriver reference.	Use ThreadLocal and remove shared mutable static driver fields.
Parallel page objects act on wrong session	BasePage has static WebDriver field or page object is shared.	Use instance `protected final WebDriver driver`; create one page object per test thread.
NoSuchSessionException	Driver session already quit or stale driver reused.	Check teardown timing and ThreadLocal.remove().
NoSuchWindowException	Window closed or wrong driver reference.	Check ownership, window handling and parallel static fields.
ParameterException	XML parameter missing or name mismatch.	Match @Parameters value to `<parameter name=...>`.
Select throws UnexpectedTagNameException	Locator is not a native select element.	Inspect DOM; use custom-dropdown interaction if needed.
ElementNotInteractableException	Element hidden, disabled, overlayed or not ready.	Wait for visibility/clickability; check overlay and locator.
Registration randomly fails on parallel run	Duplicate data or server/environment limits.	Use unique data and inspect server-side constraints.
Test never stops retrying as expected	Retry count or analyzer registration wrong.	Confirm max retry logic and XML listener class name.
Test passes only on retry	Flaky timing/environment issue.	Record first failure, add proper waits and investigate root cause.

Debugging order

1. Read the FIRST exception line and the complete stack trace.
2. Check which thread/test context failed and which browser parameter it received.
3. Log browser name, thread name and driver session ownership at setup.
4. Confirm DriverManagement.setDriver() occurs before page object creation.
5. Search for every static WebDriver and static WebElement/page field in the project.
6. Check teardown: driver.quit() followed by ThreadLocal.remove().
7. Check test data uniqueness and application-side validation messages.
8. Only then adjust waits, locators or retry policy.

17. Best practices and code-review checklist

Driver and browser management

- Use a single DriverFactory to centralize browser creation and browser options.
- Validate browser configuration before attempting to create a driver.
- Do not store a parallel test driver in one static WebDriver field.
- Use ThreadLocal only when you understand thread ownership and clean it up with remove().
- Never share a WebDriver or page object instance across parallel test threads.

- Use one browser session per independent test context, or per test method when method-level isolation is required.
- Always call driver.quit() in teardown; quit closes the complete session, while close closes only the current window.
- Capture useful logs, screenshots and browser/session information when a test fails.

Page Object Model

- Keep locators private and expose meaningful business actions through public methods.
- Keep test data and assertions in the test layer, not in the page object.
- Use one page class per meaningful page or page component; avoid enormous page classes.
- Use clear Java field names that match the UI meaning.
- Use PageFactory consistently if the project has chosen it; do not mix multiple page-object styles without a reason.
- Validate every locator against the application DOM. A syntactically valid @FindBy can still point to the wrong element.
- Use Select only for native HTML select elements.
- Avoid inheritance from BaseTest in page classes; keep BasePage separate.

Waits and synchronization

- Prefer explicit waits for meaningful conditions such as visibility, clickability, URL change or a confirmation message.
- Avoid Thread.sleep() as a normal synchronization strategy. It waits for the full fixed duration even if the condition is ready earlier.
- Keep implicit wait consistent, preferably zero when explicit waits are the framework standard.
- Do not mix large implicit and explicit waits because their effects can be difficult to reason about.

Test design and data

- Assert an observable outcome; clicking Submit alone does not prove registration succeeded.
- Generate unique usernames/emails when tests can run in parallel.
- Keep credentials out of source control. Use environment variables or a secrets manager for sensitive test secrets.
- Use synthetic test data and avoid personal data in examples or CI reports.
- Make each test independent so order and parallel scheduling do not change its result.

Retry policy

- Use a small retry limit and document the types of failures that can be retried.
- Do not use retries to mask defects or replace proper synchronization.
- Log each retry with test name, retry number and failure reason.
- Check the final report for tests that passed only after retry; treat these as flaky candidates.
- Ensure retry state is not a single static counter shared across unrelated tests.

Quick code-review checklist

- No shared static WebDriver field in parallel tests.
- DriverFactory validates the browser and returns a valid WebDriver.
- DriverManagement has set, get, get-or-null and remove operations.
- BasePage stores an instance-level final driver, not a static driver.
- Page object is created only after setup initializes the current driver.
- Every test has meaningful assertions.
- Parallel tests use independent test data.
- Teardown always attempts quit and removes ThreadLocal state.
- TestNG XML parameter and class names match the Java code.
- Retry is limited, logged and justified.

18. Hands-on exercises

Complete these tasks in order. Run the suite after each change so you can identify exactly which design change fixed or introduced a problem.

1. Create a Maven TestNG project and add the Selenium and TestNG dependencies that match your team-approved versions.
2. Create DriverFactory with Chrome, Edge and Firefox support. Add validation for null and unsupported browser names.
3. Create DriverManagement using ThreadLocal<WebDriver>. Implement setDriver(), getDriver(), getDriverOrNull() and unload().
4. Create BaseTest with @BeforeTest and @AfterTest. Log the browser name and current thread name in setup.
5. Create BasePage with `protected final WebDriver driver`. Remove `extends BaseTest` from RegisterPage.
6. Create RegisterPage using @FindBy and PageFactory.initElements(). Implement all form actions from the class source.
7. Run one registration test in Chrome through testng.xml. Add a real assertion for the post-submit result.
8. Add a second XML <test> using Firefox. Run serially and compare behavior.
9. Enable `parallel="tests"` and `thread-count="2"`. Confirm Chrome and Firefox use independent sessions.
10. Generate a unique email per run so parallel registration attempts do not conflict.
11. Add Retry and Transformer. Deliberately cause a temporary test failure and verify the maximum attempt count.
12. Review logs and the TestNG report. Identify whether any test passed only after retry.

Expected learning outcomes

- You can explain why static WebDriver is unsafe for parallel execution.
- You can explain how ThreadLocal.get(), set() and remove() work.
- You can distinguish DriverFactory from DriverManagement.
- You can explain why BasePage should not inherit BaseTest.
- You can calculate initial attempts plus retry attempts correctly.

19. Interview questions and answers

Q. What is Driver Factory design pattern?

A. It centralizes browser-driver object creation. The caller supplies a browser type and the factory returns the matching WebDriver with the required options.

Q. Why did we move browser logic from BaseTest to DriverFactory?

A. To separate object creation from test lifecycle configuration and avoid duplicating browser-selection code.

Q. What is the purpose of ChromeOptions?

A. It configures Chrome before startup, for example headless or incognito mode.

Q. What is the difference between headless and incognito?

A. Headless runs without a visible UI. Incognito starts a private browsing profile. They solve different problems and can be combined if required.

Q. Why is static WebDriver a problem in parallel execution?

A. All threads share one static field. One thread can overwrite the driver reference another thread expects to use.

Q. What is ThreadLocal?

A. A Java class that stores a separate value for each thread using the same ThreadLocal variable.

Q. Does ThreadLocal create a new WebDriver?

A. No. It only stores a value per thread. DriverFactory or another creator must create the WebDriver.

Q. Why should ThreadLocal.remove() be called?

A. To clear the value for the current thread and avoid stale references when worker threads are reused.

Q. Is a static ThreadLocal shared across threads?

A. The ThreadLocal object is shared, but each thread has its own associated value.

Q. Why should BasePage.driver not be static?

A. Page objects created by different test threads must not overwrite one shared driver field. An instance field binds the page object to its thread's driver.

Q. Why should RegisterPage not extend BaseTest?

A. Page objects belong to the page layer; BaseTest owns TestNG lifecycle and browser setup. Separating them reduces coupling.

Q. What is the purpose of PageFactory.initElements()?

A. It initializes fields annotated with PageFactory locator annotations such as @FindBy for the given page object and search context.

Q. What does @Parameters do in TestNG?

A. It injects a value from testng.xml into a configuration or test method parameter.

Q. What does parallel="tests" mean?

A. It allows the separate XML <test> contexts to execute concurrently.

Q. What does thread-count="2" mean?

A. It configures the maximum number of worker threads for parallel execution in that suite context.

Q. What is a Singleton?

A. A class design that restricts instantiation and provides one shared instance, typically returned from getInstance().

Q. Does the source getInstance() implement Singleton?

A. No. It returns `new DriverManagement()` every time, so different calls create different manager objects.

Q. What is IRetryAnalyzer?

A. A TestNG interface whose retry method returns true to request another execution or false to stop retrying.

Q. If maxCounter is 4, how many total executions can happen?

A. Five total: one initial run plus four retries.

Q. What is IAnnotationTransformer?

A. A TestNG listener that can change test annotation metadata before execution, such as attaching a retry analyzer.

Q. Why is retry not a substitute for explicit waits?

A. Retry reruns the entire test after failure; an explicit wait synchronizes a specific action with a condition during the same test attempt.

Q. Why can parallel registration fail after ThreadLocal is fixed?

A. The tests may still use duplicate account data, or the application/server may have concurrency limits.

Q. What should a complete registration test assert?

A. A verifiable application outcome such as a confirmation message, success page, created account state or other stable business result.

20. Glossary and final revision sheet

Word	Meaning
AUT	Application Under Test.
DOM	Document Object Model - the browser representation of the page structure.
Locator	A strategy to identify an element, such as id, name, CSS selector or XPath.

Word	Meaning
WebElement	Selenium representation of an element on the web page.
Page Object	A Java class that models a web page or component.
Factory	A creational design pattern that centralizes object creation.
Thread	A path of execution within a process.
ThreadLocal	A Java mechanism for associating a value with the current thread.
Race condition	A defect caused by timing-dependent access to shared mutable state.
TestNG suite	Top-level test configuration defined in XML.
Retry	A controlled additional execution after a test failure.
Flaky test	A test that produces inconsistent results without a meaningful code or product change.

The whole class in 12 statements

1. TestNG XML passes the requested browser name.
2. @Parameters injects that browser name into setup.
3. DriverFactory creates the requested browser session.
4. DriverManagement stores the session in ThreadLocal for the current thread.
5. BaseTest configures the browser and opens the application.
6. BasePage obtains the current thread's WebDriver as an instance field.
7. RegisterPage initializes @FindBy fields with PageFactory.
8. The test calls page methods with test data.
9. The test verifies a real expected application result.
10. @AfterTest quits the browser session.
11. DriverManagement.remove() clears the thread-local reference.
12. If a failure is eligible, Retry and Transformer allow a bounded number of retries.

Final takeaway Parallel Selenium execution is mainly an isolation problem. Create one browser per execution context, keep each driver thread-local, avoid static mutable driver/page state, use unique test data, and always clean up. Retry is a separate mechanism for bounded re-execution; it does not fix poor test design.

Source alignment and limitations

These notes are based on the uploaded Flow(3).txt class material. The sequence and major concepts are preserved. The notes explicitly correct the source where needed: the static WebDriver and static BasePage driver are unsafe for parallel execution; the Singleton implementation shown returns a new instance and is not a true Singleton; maxCounter=4 allows four retries after the initial run; the test should assert a result; and ThreadLocal.remove() should be called during cleanup.

The uploaded text does not contain a Maven dependency file, exact Selenium/TestNG versions, a local exception stack trace, screenshots of the registration form, or the post-registration DOM. Therefore, the exact dependency versions, exact runtime exception and success-message locator are intentionally not invented.